

Paralelismo no Cálculo de Folha de Pagamento

Simulação em Java com 300 milhões de registros em memória: comparação de desempenho entre processamento serial e paralelo, medindo **tempo de parede** e **speed-up** com 1 a 16 threads.

Aluno: Matheus C. P. Santos

RA 2609380

Curso de Análise e Desenvolvimento de Sistemas

Professor: Luís Cassiano G. Rista

O que a aplicação simula

Ciclo Anual

12 meses por empregado, com eventos mensais: faltas, afastamentos, horas extras e descontos (INSS, IRRF, FGTS).

Eventos Especiais

13º salário simplificado em dezembro, periculosidade via `Math.pow`, rescisão e demissão com regras simplificadas.

Reprodutibilidade

Sorteios determinísticos por ID e mês garantem resultados idênticos entre execuções.

Configuração do Experimento

300M

Registros

Empregados em memória

16

Threads máx.

Configurações de 1 a 16

116s

Tempo Serial

Referência base (1 thread)

15

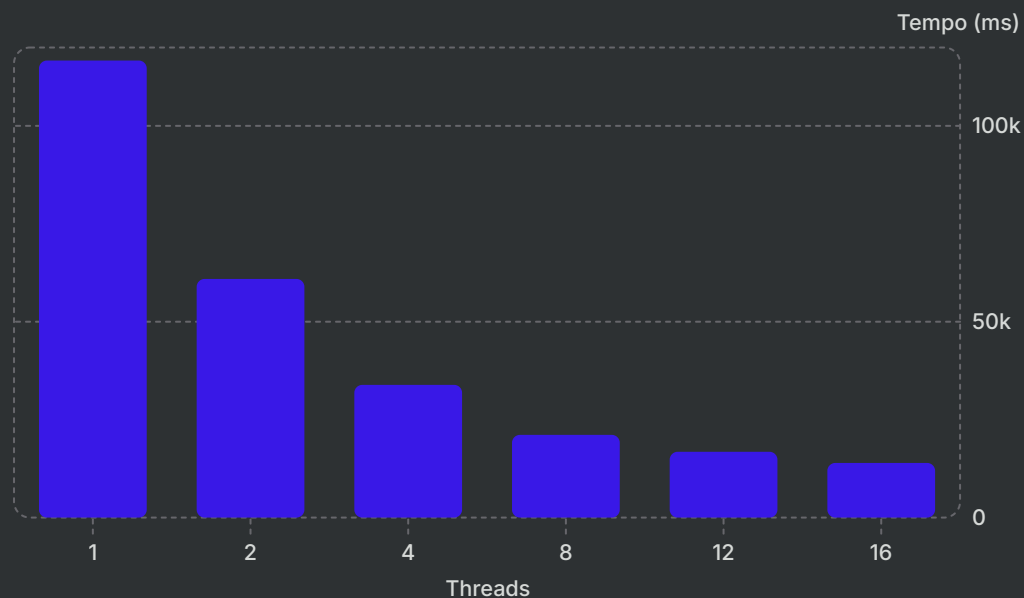
Lotes

Divisão da carga serial

RAM física: **79–82%** · Heap JVM: **~57%** do máximo · CPU do processo Java: de **~9%** (poucas threads) a **~68%** (15 threads).

Leituras pontuais de CPU ilustram tendência de uso — não substituem um profiler.

Tempos de Execução por Configuração



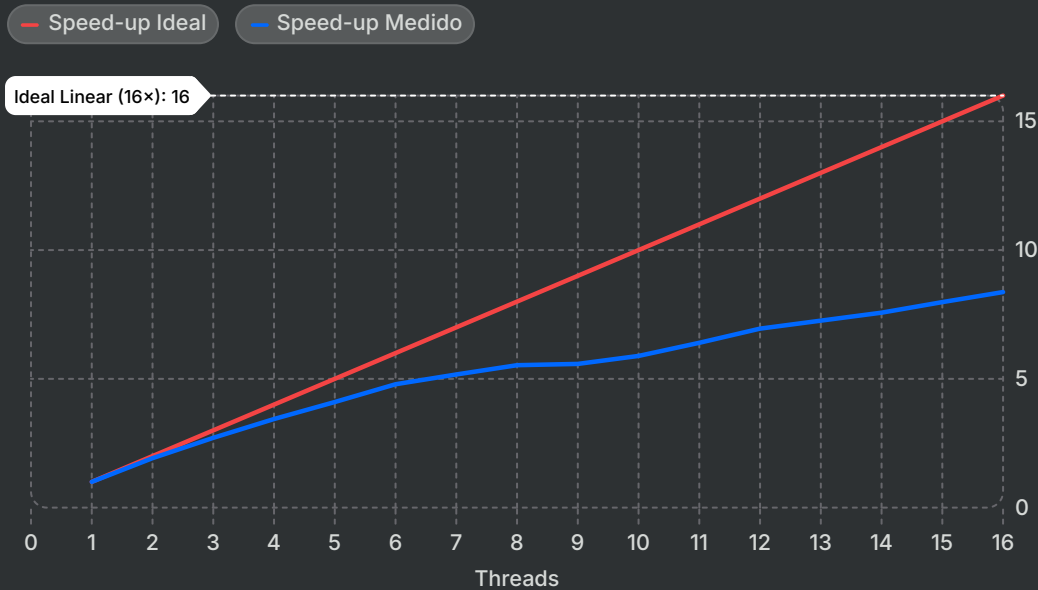
Queda consistente

O tempo cai de **116.677 ms** (1 thread) para **13.947 ms** (16 threads) — redução de **~88%**.

Não houve regressão em nenhuma configuração: adicionar threads sempre reduziu o tempo absoluto.

ⓘ ⚠ A soma dos tempos paralelos (370.155 ms) refere-se a 15 execuções independentes — **não é comparável** ao tempo de uma única execução serial.

Speed-up Medido vs. Ideal Linear



Afastamento progressivo do ideal

Com 16 threads, speed-up real = **8,37x** vs. ideal = **16x**. A diferença de **~47,7%** cresce conforme N aumenta.

Com 2 threads, eficiência de **95,76%** — quase linear. Com 16 threads, cai para **52,29%**.

Por que o Speed-up é Sublinear?

Lei de Amdahl

Trechos seriais implícitos limitam o ganho máximo, independentemente do número de threads.

Overhead de Sincronização

Coordenação entre threads (barreiras, locks, filas) consome ciclos que não existem no serial.

Contenção de Recursos

Threads competem por cache, barramento de memória e largura de banda, reduzindo eficiência.

Resultados da Simulação de Folha

Líquido Anual (300M registros)

Maior

R\$ 81.801,38

Média

R\$ 41.326,63

Menor

R\$ 489,47

Eventos (amostra de 1M empregados)

- Faltas totais: **8.798.319**
- Meses com afastamento: **175.933**
- Dias afastados: **2.285.317**
- Demissões simuladas: **47.095**

Rankings de top faltas e top descontos são curiosidades da amostra — não impactam o benchmark.

Conclusões e Trabalhos Futuros

→ Speed-up real: **8,37×** com 16 threads

Redução de ~88% no tempo absoluto — de 116 s para 14 s — comprovando ganho significativo com paralelismo.

→ Eficiência cai de 96% para 52%

Overhead, GC e trechos seriais implícitos explicam o afastamento do ideal — coerente com a Lei de Amdahl.

Ou seja, com 16 threads o tempo **cai** (melhora), só que **não melhora proporcionalmente** ao número de threads.

[Repositório no GitHub](#)